Validation

ELL7282 Assignment 2 · Dharun Thota (Entry No. 2026EET2401)

How the tests were run

Each test loads its program into IMEM, starts from RESET, sets the input pins as listed and reads the output pins: PC, HALTED, STALL, FLUSH, the forwarding selects SEL_A and SEL_B, the register-write port (WB_WE, WB_RD, WB_DATA), the store port (ST_WE, ST_ADDR, ST_DATA), the actuator pins, DISPLAY, TRIP, START_PEND and the timer pins. Register results are read from the register-write port. Inputs are changed only while the clock is low, so they are stable at every rising edge.

Group	Test	Result
1. Forwarding, load-use stall, youngest writer	T1	Pass
2. Taken branch kills wrong-path store and HALT	T2	Pass
3. RAM store/load, reserved-address isolation	T3	Pass
4. Timer: count, MATCH, clear and rearm, wraparound, stall	T4 (parts A and B)	Pass
5. Safety interlock	T5	Pass
6. End-to-end wash, two modes	washing program	Pass

Group 1: Forwarding, load-use stall and youngest-writer priority

The program sets up the registers, then runs the three hazard cases of assignment section 8.1–8.3, each separated by NOPs.

Test case

Initial inputs and state	RESET. DOOR_CLOSED = 1, all other inputs 0. Registers and RAM are set by the program (addresses 00–06).
Input changes	None.
Expected	• 8.1: SUB R3,R1 takes R1 = 0x03 from EX/MEM with no stall; R3 ← 0x00. • 8.2: STALL = 1 for exactly one cycle; ADD R3,R1 takes R1 = 0x21 from MEM/WB; R3 ← 0x21. • 8.3: ST R1,[R4] stores the youngest value 0x02 (from SUB), not the stale 0x01 or the older 0x03. • Final: R1 = 0x02, R2 = 0x02, R3 = 0x01, R4 = 0x10, R5 = 0x21, R6 = 0x05, RAM[0x05] = 0x21, RAM[0x10] = 0x02.
Observed	• SEL_B = 01 while SUB R3,R1 was in EX, STALL = 0; register write R3 ← 0x00. • STALL = 1 for one cycle only, while ADD R3,R1 was in ID; then SEL_B = 10 and R3 ← 0x21. • SEL_A = 01 for ST; store write ST_ADDR = 0x10, ST_DATA = 0x02. • Final register and RAM values as expected; HALTED = 1.
Result	**Pass**

Program

Addr	Hex	Assembly	Comment
00	9901	`LDI R1, 1`	
01	9A02	`LDI R2, 2`	
02	9B03	`LDI R3, 3`	
03	9C10	`LDI R4, 0x10`	
04	9D21	`LDI R5, 0x21`	
05	9E05	`LDI R6, 0x05`	
06	3506	`ST R5, [R6]`	RAM[5] = 0x21
07	0000	`NOP`	
08	0000	`NOP`	
09	0000	`NOP`	
0A	C102	`ADD R1, R2`	8.1: R1 = 3
0B	D301	`SUB R3, R1`	R3 = 0 (forwarded, no stall)
0C	0000	`NOP`	
0D	0000	`NOP`	
0E	0000	`NOP`	
0F	B106	`LD R1, [R6]`	8.2: R1 = 0x21
10	C301	`ADD R3, R1`	R3 = 0x21 (exactly one bubble)
11	0000	`NOP`	
12	0000	`NOP`	
13	0000	`NOP`	
14	9901	`LDI R1, 1`	8.3: stale register value 1

Addr	Hex	Assembly	Comment
15	9B01	LDI R3, 1	
16	0000	NOP	
17	0000	NOP	
18	0000	NOP	
19	C102	ADD R1, R2	R1 = 3 (older writer)
1A	D103	SUB R1, R3	R1 = 2 (youngest writer)
1B	3104	ST R1, [R4]	RAM[0x10] = 2
1C	0000	NOP	
1D	0000	NOP	
1E	0000	NOP	
1F	7000	HALT	

Group 2: Taken branch kills a wrong-path actuator store and a wrong-path HALT

R5 holds the ACT address 0xF4. The store after the first BEQZ would switch an actuator on, and the two HALTs after the second BEQZ would stop the CPU; all three are on the wrong path.

Test case

Initial inputs and state	RESET (ACT = 0x00, all actuator pins 0). DOOR_CLOSED = 1, all other inputs 0.
Input changes	None.
Expected	• Each taken BEQZ flushes the two younger instructions (FLUSH = 1 for one cycle). • The wrong-path ST never writes: ACT stays 0x00 and every actuator pin stays 0. • The wrong-path ADD never writes: R3 stays 0x05. • The wrong-path HALTs do not halt; R7 ← 0x55; HALTED becomes 1 only when the final HALT reaches EX.
Observed	• FLUSH = 1 for one cycle at each taken BEQZ (PC jumped to 0B, then to 0F), and again when the final HALT was in EX. • ST_WE stayed 0 for the whole run; VALVE_IN, HEATER, MOTOR_EN_REQ, MOTOR_DIR, PUMP_OUT, DOOR_LOCK, BUZZER all stayed 0. • No register write to R3 after its LDI; R7 ← 0x55 written; HALTED = 1 only after the HALT at address 12.
Result	**Pass**

Program

Addr	Hex	Label	Assembly	Comment
00	9A01		LDI R2, 1	
01	9B05		LDI R3, 5	
02	9C06		LDI R4, 6	
03	9DF4		LDI R5, 0xF4	ACT address
04	0000		NOP	

Addr	Hex	Label	Assembly	Comment
05	0000		NOP	
06	0000		NOP	
07	D101		SUB R1, R1	
08	490B		BEQZ R1, t1	
09	3205		ST R2, [R5]	wrong path: must not write ACT
0A	C304		ADD R3, R4	wrong path
0B	D101	t1	SUB R1, R1	
0C	490F		BEQZ R1, t2	
0D	7000		HALT	wrong path
0E	7000		HALT	wrong path
0F	9F55	t2	LDI R7, 0x55	
10	0000		NOP	
11	0000		NOP	
12	7000		HALT	

Group 3: RAM store and load, and reserved-address isolation

The program writes the first and last RAM bytes, then writes 0xFF to a reserved address (0x80), the last reserved address (0xEF) and an unimplemented peripheral address (0xF7), and reads everything back.

Test case

Initial inputs and state	RESET. DOOR_CLOSED = 1, all other inputs 0.
Input changes	None.
Expected	• R3 = 0x00: a reserved address reads 0. • R4 = 0x00: an unimplemented peripheral address reads 0. • R5 = 0xA5 and R6 = 0x3C: RAM[0x00] and RAM[0x7F] keep their values, so the writes to 0x80, 0xEF and 0xF7 changed nothing and 0x80 did not alias RAM 0x00.
Observed	Register writes R3 ← 0x00, R4 ← 0x00, R5 ← 0xA5, R6 ← 0x3C; no actuator, DISPLAY or timer output changed.
Result	**Pass**

Program

Addr	Hex	Assembly	Comment
00	99A5	LDI R1, 0xA5	
01	9A00	LDI R2, 0x00	
02	9E3C	LDI R6, 0x3C	
03	9F7F	LDI R7, 0x7F	
04	3102	ST R1, [R2]	RAM[0x00] = 0xA5
05	3607	ST R6, [R7]	RAM[0x7F] = 0x3C
06	9DFF	LDI R5, 0xFF	
07	9C80	LDI R4, 0x80	

Addr	Hex	Assembly	Comment
08	3504	`ST R5, [R4]`	reserved: ignored
09	9EEF	`LDI R6, 0xEF`	
0A	3506	`ST R5, [R6]`	reserved: ignored
0B	9EF7	`LDI R6, 0xF7`	
0C	3506	`ST R5, [R6]`	unimplemented peripheral: ignored
0D	B304	`LD R3, [R4]`	reserved read = 0
0E	B406	`LD R4, [R6]`	unimplemented read = 0
0F	B502	`LD R5, [R2]`	RAM[0] still 0xA5
10	B607	`LD R6, [R7]`	RAM[0x7F] still 0x3C
11	7000	`HALT`	

Group 4: Timer

Part A runs two timer intervals with a clear and rearm between them, then halts while the clock keeps running. Part B shows that the timer keeps counting during a load-use stall.

Test case

Initial inputs and state	RESET (timer disabled, COUNT = LIMIT = 0, MATCH = 0). DOOR_CLOSED = 1, all other inputs 0.
Input changes	None. In Part A the clock keeps running after HALT until COUNT has passed 0xFF.
Expected	• A: TIMER_MATCH = 1 when TIMER_COUNT reaches 0x05 (LIMIT = 5). • A: the second store of 0x03 to TIMER_CTRL clears COUNT and MATCH at the same edge, although MATCH was 1. • A: MATCH = 1 again when COUNT reaches 0x04 (LIMIT = 4). • A: after HALT the timer keeps counting; COUNT goes 0xFF → 0x00 and MATCH stays 1. • B: R3 = 0x03: three timer ticks between two samples that are two instructions apart, because one stall bubble passed between them.
Observed	• A: MATCH = 1 with COUNT = 0x05; after the second clear COUNT = 0x00 and MATCH = 0; MATCH = 1 with COUNT = 0x04. • A: HALTED = 1 while COUNT kept rising; COUNT 0xFF → 0x00 with MATCH = 1. • B: STALL = 1 after each LD (two stall cycles); register write R3 ← 0x03.
Result	**Pass**

Program, part A

Addr	Hex	Label	Assembly	Comment
00	9FF9		`LDI R7, 0xF9`	
01	9905		`LDI R1, 5`	
02	3107		`ST R1, [R7]`	LIMIT = 5
03	9FFA		`LDI R7, 0xFA`	
04	9903		`LDI R1, 3`	

Addr	Hex	Label	Assembly	Comment
05	3107		ST R1, [R7]	EN | CLR
06	9EFB		LDI R6, 0xFB	
07	B206	p1	LD R2, [R6]	
08	9B01		LDI R3, 1	
09	E203		AND R2, R3	
0A	4A07		BEQZ R2, p1	wait for MATCH
0B	9FF9		LDI R7, 0xF9	
0C	9904		LDI R1, 4	
0D	3107		ST R1, [R7]	LIMIT = 4
0E	9FFA		LDI R7, 0xFA	
0F	9903		LDI R1, 3	
10	3107		ST R1, [R7]	clear and rearm
11	B206	p2	LD R2, [R6]	
12	9B01		LDI R3, 1	
13	E203		AND R2, R3	
14	4A11		BEQZ R2, p2	
15	7000		HALT	the timer keeps counting after HALT and wraps 255 -> 0

Program, part B

Addr	Hex	Assembly	Comment
00	9FFA	LDI R7, 0xFA	
01	9901	LDI R1, 1	
02	3107	ST R1, [R7]	enable, no clear
03	9EF8	LDI R6, 0xF8	
04	B206	LD R2, [R6]	sample A
05	C402	ADD R4, R2	load-use: one bubble
06	B306	LD R3, [R6]	sample B
07	D302	SUB R3, R2	R3 = B - A = 3
08	7000	HALT	

Group 5: Safety interlock

The program requests the motor, waits until it sees TRIP, drops the request, waits for a new START edge, acknowledges it, and requests the motor again.

Test case

Initial inputs and state	RESET. DOOR_CLOSED = 1, START = 0, all other inputs 0.
Input changes	• After MOTOR_EN_REQ = 1: DOOR_CLOSED = 0. • After the program drops the request (MOTOR_EN_REQ = 0): DOOR_CLOSED = 1. • Then START = 1.
Expected	• Motor requested: MOTOR_EN_REQ = MOTOR_EN_ACTUAL = 1. • Door opened: MOTOR_EN_ACTUAL = 0 immediately, without waiting for a clock edge; TRIP = 1 at the next edge. • Door closed again: MOTOR_EN_ACTUAL stays 0, no automatic restart (TRIP = 1). • New START edge: START_PEND = 1; after the program acknowledges, TRIP = 0 and START_PEND = 0. • Motor requested again: MOTOR_EN_REQ = MOTOR_EN_ACTUAL = 1.
Observed	• MOTOR_EN_REQ = MOTOR_EN_ACTUAL = 1 after the first ACT write. • MOTOR_EN_ACTUAL dropped to 0 at the moment DOOR_CLOSED was set to 0, while MOTOR_EN_REQ was still 1; TRIP = 1 after the next clock edge. • The program cleared MOTOR_EN_REQ; after DOOR_CLOSED = 1, MOTOR_EN_ACTUAL stayed 0 and TRIP stayed 1. • START = 1 gave START_PEND = 1; after the acknowledge store TRIP = 0 and START_PEND = 0. • MOTOR_EN_REQ = MOTOR_EN_ACTUAL = 1; HALTED = 1.
Result	**Pass**

Program

Addr	Hex	Label	Assembly	Comment
00	9FF4		`LDI R7, 0xF4`	
01	9924		`LDI R1, 0x24`	
02	3107		`ST R1, [R7]`	request motor
03	9EF0	loop1	`LDI R6, 0xF0`	
04	B206		`LD R2, [R6]`	
05	9B20		`LDI R3, 0x20`	TRIP bit
06	E203		`AND R2, R3`	
07	4A03		`BEQZ R2, loop1`	wait until TRIP is seen
08	9920		`LDI R1, 0x20`	
09	3107		`ST R1, [R7]`	clear the motor request (lock kept)
0A	B206	loop2	`LD R2, [R6]`	
0B	9B10		`LDI R3, 0x10`	START_PEND bit
0C	E203		`AND R2, R3`	
0D	4A0A		`BEQZ R2, loop2`	wait for a new START edge
0E	9DF6		`LDI R5, 0xF6`	
0F	9901		`LDI R1, 0x01`	
10	3105		`ST R1, [R5]`	ACK_START: rearm
11	9924		`LDI R1, 0x24`	
12	3107		`ST R1, [R7]`	request the motor again

Group 6: End-to-end wash in two modes

The washing program of listing.pdf (`program.hex`) is run twice from RESET, once in each mode. The water level is raised step by step while VALVE_IN = 1 and lowered while PUMP_OUT = 1; DRUM_RPM follows the motor and returns to 0 a few cycles after it stops.

	LIGHT (MODE = 00)	**HEAVY (MODE = 01)**
Initial inputs	RESET. DOOR_CLOSED = 1, START = 0, WATER_LEVEL = 0, TEMP = 0x19, DRUM_RPM = 0.	
Start	START = 1; after DOOR_LOCK = 1, START = 0 and MODE changed to 01	START = 1; after DOOR_LOCK = 1, START = 0 and MODE changed to 00
Fill (WATER_LEVEL)	20, 40, 60, 80, 99, 102 (target 100 is skipped)	40, 80, 120, 160, 179, 185 (target 180 is skipped)
Drain (WATER_LEVEL)	70, 40, 12, 5 (threshold 8 is skipped)	140, 90, 40, 9, 3 (threshold 8 is skipped)
Expected	Lock; fill until level $\geq$ 100 (VALVE_IN on at 99, off at 102); wash 2 forward/reverse cycles of 10 ticks each way; drain until level $\leq$ 8; spin 12 ticks; stop; unlock when DRUM_RPM = 0; DISPLAY = 0x0D, BUZZER = 1, HALTED = 1	The same with target 180 (on at 179, off at 185), 3 wash cycles of 16 ticks each way and a 24-tick spin
Observed	DISPLAY 01, 02, 03, 04, 05, 0D in order; VALVE_IN stayed 1 at 99 and went 0 at 102; 2 forward/reverse cycles on MOTOR_DIR; PUMP_OUT stayed 1 at 12 and stopped draining at 5; DOOR_LOCK = 0 only after DRUM_RPM = 0; final BUZZER = 1, all other actuators 0, HALTED = 1. The MODE change was ignored.	DISPLAY 01, 02, 03, 04, 05, 0D in order; VALVE_IN stayed 1 at 179 and went 0 at 185; 3 forward/reverse cycles on MOTOR_DIR; PUMP_OUT stayed 1 at 9 and stopped draining at 3; DOOR_LOCK = 0 only after DRUM_RPM = 0; final BUZZER = 1, all other actuators 0, HALTED = 1. The MODE change was ignored.
Result	**Pass**	**Pass**